



Lab 01: Terraform Fundamentals

Build Infrastructure with Terraform | GSP156

Overview

Used Terraform and Gemini Code Assist to provision a VM instance on Google Cloud. Learned how Terraform uses declarative configuration files to safely create and manage infrastructure.

Key Terraform Commands

Command	Purpose
<code>terraform init</code>	Initialize working directory and download providers
<code>terraform plan</code>	Preview changes without applying them
<code>terraform apply</code>	Create or update infrastructure
<code>terraform show</code>	Inspect current state of infrastructure
<code>terraform destroy</code>	Destroy all managed infrastructure

Task 1 - Verify Terraform Installation

```
terraform
```

```
CLOUD SHELL
Terminal (qwiklabs-gcp-04-c5e29464b53b) x +
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to qwiklabs-gcp-04-c5e29464b53b.
Use "gcloud config set project [PROJECT_ID]" to change to a different project.
student_0038af1edc78@cloudshell:~ (qwiklabs-gcp-04-c5e29464b53b) $ terraform
Usage: terraform [global options] <subcommand> [args]

The available commands for execution are listed below.
The primary workflow commands are given first, followed by
less common or more advanced commands.

Main commands:
  init      Prepare your working directory for other commands
  validate  Check whether the configuration is valid
  plan      Show changes required by the current configuration
  apply     Create or update infrastructure
  destroy   Destroy previously-created infrastructure

All other commands:
  console   Try Terraform expressions at an interactive command prompt
  fmt       Reformat your configuration in the standard style
  force-unlock Release a stuck lock on the current workspace
  get       Install or upgrade remote Terraform modules
  graph     Generate a Graphviz graph of the steps in an operation
  import    Associate existing infrastructure with a Terraform resource
  login     Obtain and save credentials for a remote host
  logout    Remove locally-stored credentials for a remote host
  metadata  Metadata related commands
  output    Show output values from your root module
  providers Show the providers required for this configuration
  refresh   Update the state to match remote systems
  show      Show the current state or a saved plan
  state     Advanced state management
  taint     Mark a resource instance as not fully functional
  test      Experimental support for module integration testing
  untaint   Remove the 'tainted' state from a resource instance
  version   Show the current Terraform version
  workspace Workspace management

Global options (use these before the subcommand, if any):
  -chdir=DIR Switch to a different working directory before executing the
              given subcommand.
  -help     Show this help output, or the help for a specified subcommand.
  -version  An alias for the "version" subcommand.
```

Task 2 - Build Infrastructure with Gemini Code Assist

Enabled Gemini Code Assist in Cloud Shell IDE, created instance.tf, and used Gemini to generate a Terraform configuration for a VM.

Prompt used in Gemini Code Assist:

```
Generate the Terraform configuration for a Google Compute Engine virtual machine
* Project ID: [PROJECT_ID]
* VM Name: terraform
* Machine Type: e2-medium
* Deployment Zone: [ZONE]
* Boot Disk: Debian image
* Network: Default network
```

Generated instance.tf:

```
resource "google_compute_instance" "default" {
  project      = "[PROJECT_ID]"
  zone         = "[ZONE]"
  name         = "terraform"
  machine_type = "e2-medium"
  boot_disk {
    initialize_params { image = "[DEBIAN_IMAGE]" }
  }
  network_interface { network = "default" }
}
```

```
terraform init
terraform plan
terraform apply
terraform show
```

The screenshot shows the Google Cloud Platform console's 'VM instances' page. At the top, there are buttons for 'Create instance', 'Import VM', 'Refresh', and a 'Learn' link. Below these are tabs for 'Instances', 'Observability', and 'Instance schedules'. The 'Instances' tab is active, showing a table of VM instances. The table has columns for Status, Name, Zone, Recommendations, In use by, Internal IP, External IP, and Connect. A single instance named 'terraform' is listed in the 'europa-west1-d' zone, with an internal IP of 10.132.0.2 and an external IP of 34.38.99.44. The 'Connect' column shows 'SSH' and a dropdown menu. Below the table, there is a 'Related actions' section with a 'Hide' link.

Status	Name	Zone	Recommendations	In use by	Internal IP	External IP	Connect
☒	terraform	europa-west1-d			10.132.0.2 (nic0)	34.38.99.44 (nic0)	SSH

```

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
student_00.5b8f12edia7c@cloudshell:~ (qwiklabs-gcp-04-c5e29464b53b)$ terraform show
# google_compute_instance.terraform:
resource "google_compute_instance" "terraform" {
  can_ip_forward      = false
  cpu_platform        = "Intel Broadwell"
  creation_timestamp   = "2026-06-11T20:27:17.892-07:00"
  current_status      = "RUNNING"
  deletion_policy      = "DELETE"
  deletion_protection = false
  effective_labels     = {
    "goog-terraform-provisioned" = "true"
  }
  enable_display      = false
  id                  = "projects/qwiklabs-gcp-04-c5e29464b53b/zones/europe-west1-d/instances/terraform"
  instance_id         = "446048956766929532"
  label_fingerprint   = "v4sU5-42LMx"
  machine_type        = "e2-medium"
  metadata_fingerprint = "GditU7kiMbk="
  name                = "terraform"
  project              = "qwiklabs-gcp-04-c5e29464b53b"
  self_link            = "https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-04-c5e29464b53b/zones/europe-west1-d/instances/terraform"
  tags_fingerprint    = "42WwSp88rSM="
  terraform_labels    = {
    "goog-terraform-provisioned" = "true"
  }
  zone                = "europe-west1-d"

  boot_disk {
    auto_delete = true
    device_name = "persistent-disk-0"
    force_attach = false
    guest_os_features = [
      "GUEST_COMPATIBLE",
      "VIRTIO_SCSI_MULTIQUEUE",
      "GVNIC",
      "SEV_CAPABLE",
      "SEV_LIVE_MIGRATABLE_V2",
    ]
    mode = "READ_WRITE"
    source = "https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-04-c5e29464b53b/zones/europe-west1-d/disks/terraform"

    initialize_params {
      architecture = "X86_64"
      enable_confidential_compute = false
      image         = "https://www.googleapis.com/compute/v1/projects/debian-cloud/global/images/debian-12-bookworm-v20260609"
      labels       = {}
      provisioned_iops = 0
      provisioned_throughput = 0
      resource_policies = {}
      size           = 10
      type            = "pd-standard"
    }
  }
}

```

What I Learned

- Terraform uses declarative .tf files to describe desired infrastructure state
- terraform init downloads the provider plugin needed for the configuration
- terraform plan shows what will change without making any changes
- terraform apply creates real infrastructure matching the configuration
- terraform.tfstate tracks the mapping between configuration and real resources
- Gemini Code Assist can generate Terraform configurations from natural language prompts

Resources

Terraform Docs: <https://developer.hashicorp.com/terraform/docs> | Lab ID: GSP156